

This line has created a `TfidfVectorizer()` function, which is capable of creating TFIDF vectors from sets of documents. To create the vectorizer, we have to specify an `ngram_range`. This tells the vectorizer what to treat as a term. We specified `(1, 1)`, meaning that our vectorizer will treat only 1-grams (individual words) as terms. If we had specified `(1, 3)`, it would treat 1-grams (single words), 2-grams (two-word phrases), and 3-grams (three-word phrases) as terms and create a TFIDF element for each of them. We also specified a `tokenizer`, for which we specified the `normalize()` function we created before. Finally, we have to specify `stop_words`, the words that we want to filter out because they're not informative. In English, stop words include *the*, *and*, *of*, and other extremely common words. By specifying `stop_words = 'english'`, we're telling our vectorizer to filter out the built-in set of English stop words and vectorize only less common, more informative words.

Now, let's configure what our chatbot will be able to talk about. Here, it will be able to talk about the chapters of this book, so we'll create a list that contains very simple descriptions of each chapter. In this context, each string will be one of our *documents*.

```
alldocuments = ['Chapter 1. The algorithmic approach to problem solving,
including Galileo and baseball.',
                'Chapter 2. Algorithms in history, including magic squares, Russian
peasant multiplication, and Egyptian methods.',
                'Chapter 3. Optimization, including maximization, minimization, and
the gradient ascent algorithm.',
                'Chapter 4. Sorting and searching, including merge sort, and
algorithm runtime.',
                'Chapter 5. Pure math, including algorithms for continued fractions
and random numbers and other mathematical ideas.',
                'Chapter 6. More advanced optimization, including simulated annealing
and how to use it to solve the traveling salesman problem.',
                'Chapter 7. Geometry, the postmaster problem, and Voronoi
triangulations.',
                'Chapter 8. Language, including how to insert spaces and predict
phrase completions.',
                'Chapter 9. Machine learning, focused on decision trees and how to
predict happiness and heart attacks.',
                'Chapter 10. Artificial intelligence, and using the minimax algorithm
to win at dots and boxes.',
```